# SIMATS ENGINEERING
## ASSESSMENT TOOL 3

**COURSE NAME: INTERNET PROGRAMMING**
**COURSE CODE: CSA4303**
**Name: Duddela Harika**
**Reg No: 192324301**
**Total Marks: 96**

---

**COURSE OUTCOME COVERED**

**CO-1:** Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

| | |
|---|---|
| Assessment Tool: | Code along exercise: Make your first HTML page together with CSS |

**Weightage: 10%**

---

**QUESTIONS:** **Total Marks: 10**

## Scenario: Web Development Standards Implementation

A start up is developing a modern web application. During development, the team encounters multiple issues related to HTML elements, HTTP methods, XML syntax, and web communication protocols.
The following observations are made:

- The team needs to embed a promotional video on the homepage.
- A registration form must securely send user data to the server.
- The navigation menu must follow semantic HTML standards.
- XML configuration files are being used but contain syntax errors.
- The application must communicate over the correct web protocol.

## Data Snippets Provided

### Snippet 1: Video Embedding Options

```
A. <media src="video.mp4"></media>
B. <video src="video.mp4" controls></video>
```

```
C. <movie src="video.mp4"></movie>
D. <vid src="video.mp4"></vid>
```

**Snippet 2: Form Submission Methods**

```
A. GET
B. POST
C. PUT
D. DELETE
```

**Snippet 3: Navigation Elements**

```
A. <div>
B. <nav>
C. <section>
D. <header>
```

**Snippet 4: XML Syntax**

```
A. <note><to>User</to><from>Admin</from>
B. <note><to>User</to><from>Admin</from></note>
C. <note><to>User</from></note>
D. <note to="User"></note
```

**Snippet 5: Web Communication Protocols**

```
A. FTP
B. HTTP
C. SMTP
D. SNMP
```

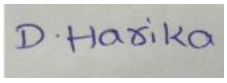
## Questions:

1. Select the **correct HTML tag for video embedding** and justify your answer.
2. Identify the **appropriate HTTP method for form submission** and explain why.
3. Choose the **proper semantic element for navigation** and justify its usage.
4. Identify the **correct XML syntax** and explain the error in other options.
5. Choose the **correct protocol for web communication** and justify your choice.

---

***Code of Conduct:***

*I **Duddela Harika / 192324301** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:*** 

## RUBRICS FOR CALCULATION:

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning | Marks |
|---|---|---|---|---|---|---|
| Understanding of Case | 25 | Thorough understanding; clearly identifies all key issues | Good understanding with most issues identified | Basic understanding; some issues missed | Poor understanding; problem not identified correctly | 24 |
| Application of Concepts | 25 | Effectively applies relevant concepts to analyze the case deeply | Applies appropriate concepts with minor gaps | Limited application of concepts | Fails to apply relevant concepts | 24 |
| Analysis | 20 | In-depth analysis with strong reasoning and insights | Good analysis with logical reasoning | Basic analysis with limited depth | Weak or no analysis; lacks reasoning | 19 |
| Solution Design | 20 | Innovative, feasible solutions with strong justification | Logical solutions with adequate justification | Basic solutions with weak justification | Incorrect or no solution provided | 19 |
| Clarity | 10 | Clear, well-structured, and easy to understand | Organized and understandable presentation | Limited clarity and structure | Poor clarity; difficult to understand | 10 |

**Total : 96**

## Answers:

**Question 1: Select the correct HTML tag for video embedding and justify your answer.**

**Correct Answer:** Option B — <video src="video.mp4" controls></video>

HTML (HyperText Markup Language) is the standard markup language used to create webpages. It provides different elements for displaying text, images, audio, video, links, forms, and other content.

In the given scenario, the startup wants to display a promotional video on its homepage. HTML5 provides the <video> element to embed video content directly into a webpage without requiring external plugins.

**Explanation of the correct answer**

The correct option is:

<video src="video.mp4" controls></video>

The <video> element allows the browser to display and play a video file. The src attribute specifies the location of the video, while the controls attribute displays the built-in video controls.

The user can use these controls to play, pause, adjust the volume, and manage playback.

**HTML code**

```
<!DOCTYPE html>
<html>
<head>
   <title>Promotional Video</title>
</head>
<body>
   <h1>Welcome to Our Startup</h1>
   <p>Watch our promotional video to learn about our services.</p>

   <video width="500" controls>
      <source src="video.mp4" type="video/mp4">
      Your browser does not support the video element.
   </video>

</body>
</html>
```

**Explanation of the HTML attributes**

- <video>: Defines the video player.
- src: Specifies the video file location when used directly on the video element.
- <source>: Specifies a video source and its media type.
- type="video/mp4": Indicates that the video is in MP4 format.
- controls: Displays the browser's built-in playback controls.
- width="500": Sets the video player's width to 500 pixels.
- Fallback text: Displays a message if the browser cannot use the video element.

**Why are the other options incorrect?**

| Option | HTML element | Explanation |
|---|---|---|
| A | <media> | Not a standard HTML element for embedding video. |
| B | <video> | Correct standard HTML element for embedding video. |
| C | <movie> | Not a standard HTML video element. |
| D | <vid> | Not a standard HTML video element. |

**Advantages of using the <video> element**

1. It embeds videos directly into webpages.
2. It provides built-in playback controls.
3. It does not require a separate browser plugin.
4. It supports commonly used video formats, depending on the browser.
5. It improves the presentation of promotional and educational content.
6. It supports additional attributes such as autoplay, muted, loop, and poster.

**Conclusion**

The <video> element is the correct choice because it is a standard HTML5 element designed for displaying video content. Using it allows the startup to present its promotional video directly on the homepage and provide visitors with convenient playback controls.

## Question 2: Identify the appropriate HTTP method for form submission and explain why.

Correct Answer: Option B — POST

HTTP stands for Hypertext Transfer Protocol. It is an application-layer protocol that enables communication between a client, such as a web browser, and a web server.

Web applications use HTTP methods to indicate what action should be performed on the server. Common methods include GET, POST, PUT, and DELETE.

In the given scenario, the startup needs a registration form that sends user information to the server. Since the form may contain sensitive information such as usernames, email addresses, and passwords, the POST method is the most appropriate choice among the given options.

**Explanation of the correct answer**

The POST method submits data to a server through the body of an HTTP request. Unlike GET, it does not normally place the submitted form values directly in the URL.

When a user fills out the registration form and clicks the Register button, the browser sends the entered information to the server. The server can then validate the information, create an account, and return a response.

Important security point: POST is suitable for submitting registration data, but it does not encrypt the data by itself. HTTPS must be used to protect information during transmission. The application should also validate inputs, hash passwords securely, and use appropriate protections against attacks.

**HTML code**

```
<!DOCTYPE html>
<html>
<head>
```

```
    <title>Registration Form</title>
</head>
<body>
    <h1>User Registration</h1>

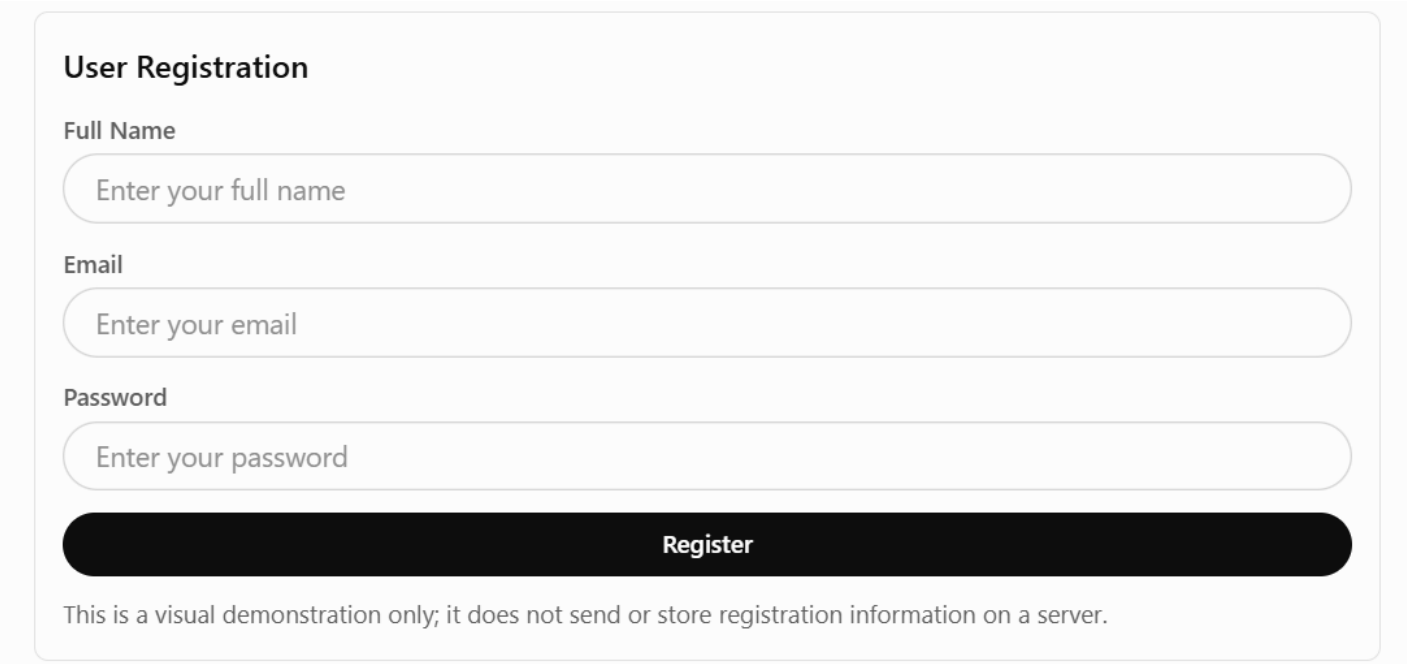
    <form action="/register" method="POST">
        <label for="name">Full Name:</label>
        <input type="text" id="name" name="fullname" required>
        <br><br>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>
        <br><br>

        <label for="password">Password:</label>
        <input type="password" id="password"
            name="password" required>
        <br><br>

        <button type="submit">Register</button>
    </form>
</body>
</html>
```

**Output**




**Working procedure**

The form submission process takes place in the following steps:

1. The user opens the registration webpage.
2. The user enters their name, email address, and password.
3. The browser checks required fields and basic input validity.
4. The user clicks the Register button.

5. The browser sends an HTTP POST request to /register.
6. The server receives the request and validates the submitted information.
7. The server securely processes the registration and returns an appropriate response.

The server endpoint must exist for the registration process to work. HTML alone does not create an account or save information in a database.

**Comparison of HTTP methods**

| HTTP method | Purpose | Example |
|---|---|---|
| GET | Retrieve information | Display a list of users |
| POST | Submit data or create a resource | Register a new user |
| PUT | Create or replace a resource at a known URL | Replace a user profile |
| DELETE | Remove a resource | Delete an account |

**Why are the other options less appropriate?**

Option A — GET: GET is primarily intended to retrieve information. When form values are submitted through a typical GET request, they appear in the URL query string. This is unsuitable for passwords and other confidential information.

Option B — POST: Correct. POST sends form data in the request body and is appropriate for submitting registration information.

Option C — PUT: PUT is commonly used to create or replace a resource at a specified URL. It is not the usual choice for a standard HTML registration form.

Option D — DELETE: DELETE is used to request removal of a resource, not to register a new user.

**Security measures for registration forms**

A secure registration system should include:

- HTTPS encryption to protect data in transit.
- Server-side validation of all submitted fields.
- Secure password hashing instead of storing plain-text passwords.
- Protection against SQL injection through parameterized database queries.
- Appropriate session management and access controls.
- CSRF protection where applicable.
- Clear error messages that do not reveal sensitive internal details.

**Conclusion**

POST is the most appropriate option for the registration form because it is designed for submitting data to a server. When combined with HTTPS, server-side validation, secure password storage, and suitable application protections, it supports a safer and more reliable registration process.

## Question 3: Choose the proper semantic element for navigation and justify its usage.

Correct Answer: Option B — <nav>

Semantic HTML refers to the use of HTML elements that clearly describe the meaning and purpose of their content. Semantic elements help developers organize webpages and make their structure easier to understand.

Examples of semantic HTML elements include <header>, <nav>, <main>, <section>, <article>, and <footer>.

In the given scenario, the startup wants its navigation menu to follow semantic HTML standards. Therefore, the <nav> element is the correct choice.

**Explanation of the correct answer**

The <nav> element represents a section of a webpage that contains important navigation links. These links help users move between different pages or sections of a website.

A typical startup website may include links such as:

- Home
- About Us
- Services
- Products
- Contact Us

The <nav> element identifies this collection of links as navigation rather than ordinary content.

**HTML code**

```
<!DOCTYPE html>
<html>
<head>
   <title>Startup Website</title>
   <style>
      body {
         font-family: Arial, sans-serif;
         margin: 0;
         background-color: #f2f6fc;
      }

      header {
```

```
        background-color: #174a7e;
        color: white;
        padding: 20px;
        text-align: center;
      }

      nav {
        background-color: #263746;
        padding: 15px;
        text-align: center;
      }

      nav a {
        color: white;
        text-decoration: none;
        margin: 15px;
      }

      main {
        padding: 25px;
      }
    </style>
</head>
<body>

    <header>
      <h1>Our Startup</h1>
      <p>Innovation for a Better Future</p>
    </header>

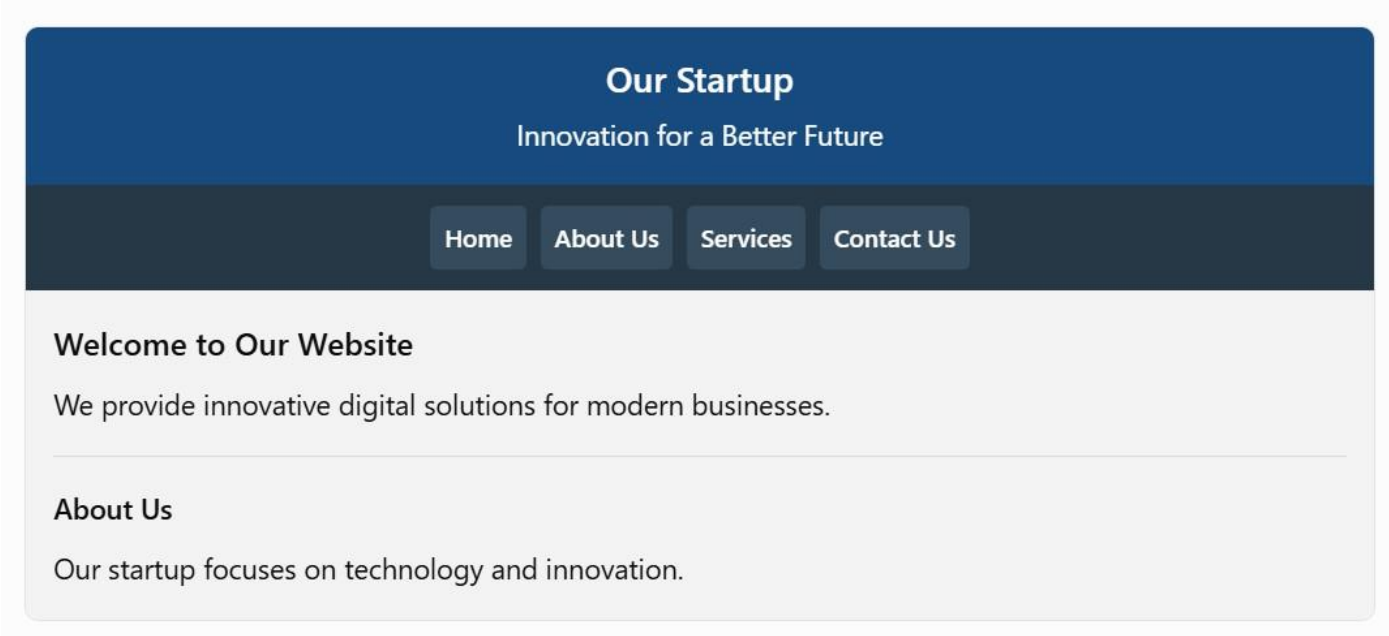
    <nav aria-label="Main navigation">
      <a href="#home">Home</a>
      <a href="#about">About Us</a>
      <a href="#services">Services</a>
      <a href="#contact">Contact Us</a>
    </nav>

    <main>
      <section id="home">
        <h2>Welcome to Our Website</h2>
        <p>
          We provide innovative digital solutions
          for modern businesses.
        </p>
      </section>

      <section id="about">
        <h2>About Us</h2>
        <p>Our startup focuses on technology and innovation.</p>
      </section>
    </main>

</body>
</html>
```

**output:**




**Advantages of the <nav> element**

a. Better accessibility: Screen readers can identify navigation landmarks, making it easier for users who rely on assistive technology to find important links.

b. Improved code readability: Developers can quickly identify the navigation section without examining every individual link.

c. Better document structure: Semantic elements clearly distinguish navigation from headings, articles, and other webpage content.

d. Easier maintenance: Developers can style or modify the navigation menu without affecting unrelated sections.

e. Search engine understanding: Semantic markup can help search engines interpret the structure and purpose of webpage content, although using <nav> alone does not guarantee better rankings.

**Why are the other options incorrect?**

| Option | Element | Purpose |
|---|---|---|
| A |  | A generic container without inherent semantic meaning |
| B | <nav> | Defines a navigation section containing important links |
| C | <section> | Groups related content into a thematic section |
| D | <header> | Contains introductory content, such as a page title or logo |

A  could technically contain navigation links, but it would not communicate their navigation purpose

as clearly as <nav>. Similarly, <header> may contain a navigation menu, but it does not replace the semantic role of <nav>.

**Best practices for semantic navigation**

1. Use <nav> for major navigation blocks rather than every small collection of links.
2. Use meaningful link text that explains where each link leads.
3. Use valid href values for navigation links.
4. Provide an accessible label, such as aria-label="Main navigation", when multiple navigation landmarks need to be distinguished.
5. Ensure that links are keyboard accessible and have visible focus indicators.
6. Use CSS to style the navigation without changing its semantic meaning.

**Conclusion**

The <nav> element is the correct semantic HTML element because it identifies a section of the webpage containing important navigation links. It improves code organization, accessibility, and maintainability while following modern HTML standards.

## Question 4: Identify the correct XML syntax and explain the error in other options.

**Correct Answer:** Option B — <note><to>User</to><from>Admin</from></note>

**Introduction**

XML stands for Extensible Markup Language. It is a markup language used to store, organize, and transport structured data. XML allows developers to create their own tags according to the requirements of an application.

In the given scenario, the startup uses XML configuration files to store application settings. However, some files contain syntax errors. Incorrect XML syntax can prevent software from reading configuration data properly and may cause application errors.

Therefore, developers must ensure that XML documents are well-formed and follow the required syntax rules.

**Explanation of the correct answer**

The correct option is:

```
<note>
  <to>User</to>
  <from>Admin</from>
</note>
```

This XML document is well-formed because:

1. It contains one root element, <note>.
2. Every opening tag has a matching closing tag.
3. The elements are properly nested.

4. The closing tags appear in the correct order.
5. The document contains valid XML element structure.

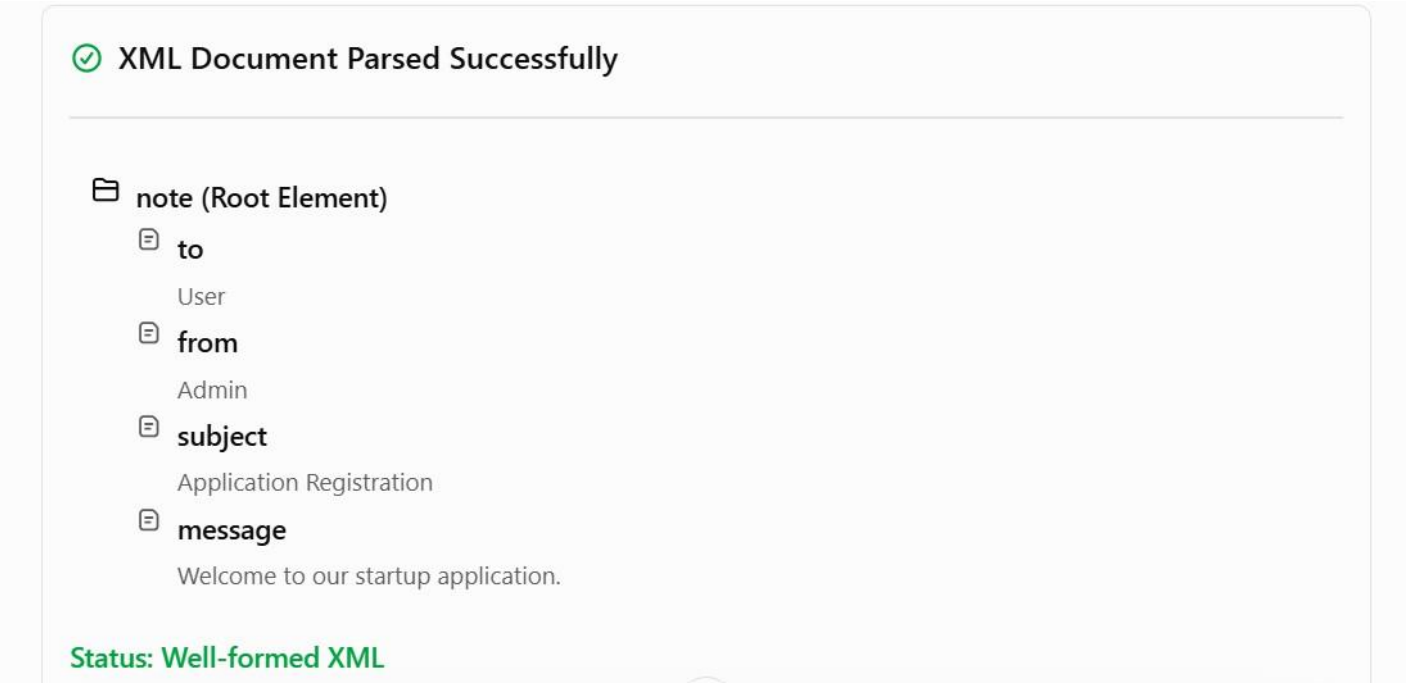
The <note> element is the root element, while <to> and <from> are child elements.

**XML code**

```
<?xml version="1.0" encoding="UTF-8"?>

<note>
   <to>User</to>
   <from>Admin</from>
   <subject>Application Registration</subject>
   <message>Welcome to our startup application.</message>
</note>
```

**Output**




**Explanation of the errors in the other options**

Option A

<note><to>User</to><from>Admin</from>

Error: The root element <note> is not closed. The document is incomplete because it does not contain the closing tag </note>.

Correction:

<note><to>User</to><from>Admin</from></note>

Option B

<note><to>User</to><from>Admin</from></note>

Status: Correct. Every element has a matching closing tag, and all elements are properly nested.

Option C

<note><to>User</from></note>

Error: The opening tag <to> is incorrectly closed with </from>. XML requires matching opening and closing tag names.

Correction:

<note><to>User</to></note>

Option D

<note to="User"></note

Error: The final closing tag is malformed because it lacks the closing angle bracket >. The syntax should be completed as follows:

<note to="User"></note>

Here, to="User" is an attribute of the root element rather than a separate child element.

**Important XML syntax rules**

| Rule | Explanation | Example |
|---|---|---|
| Single root element | A document must have exactly one root element | <note>...</note> |
| Matching tags | Opening and closing names must match | <to>User</to> |
| Proper nesting | Child elements must close in the correct order | <a>Text</a> |
| Case sensitivity | XML tag names are case-sensitive | <Name> differs from <name> |
| Quoted attributes | Attribute values must be enclosed in quotes | <user id="101"/> |
| Special characters | Reserved characters must be escaped where required | & represents & |

**Importance of XML in web applications**

XML is useful for:

- Storing application configuration settings.
- Transferring structured information between systems.
- Representing data in a standardized format.
- Supporting integrations with systems that use XML-based APIs.

- Organizing hierarchical information using parent and child elements.
- Defining data formats for certain enterprise applications and web services.

It is important to note that a well-formed XML document follows XML syntax rules. A document may still fail validation against a particular XML schema if its content does not meet that schema's requirements.

**Conclusion**

Option B is correct because it follows XML syntax rules, including matching tags, proper nesting, and a single root element. Correct XML syntax allows configuration files to be parsed reliably and reduces errors in the startup's web application.

## Question 5: Choose the correct protocol for web communication and justify your choice.

**Correct Answer:** Option B — HTTP

A web application requires a communication protocol to exchange information between a client and a server. A client is typically a web browser, while a server stores application resources, processes requests, and returns responses.

HTTP stands for Hypertext Transfer Protocol. It is an application-layer protocol used to transfer information between web clients and web servers.

In the given scenario, the startup must select the appropriate protocol for its web application. HTTP is the correct answer because it supports requests for webpages and other web resources.

In modern applications, HTTPS is generally preferred because it protects communication using TLS encryption.

**Explanation of the correct answer**

HTTP follows a request-response model.

- Client: Sends a request for a webpage or resource.
- Web server: Receives the request and processes it.
- Server response: Returns the requested content or an appropriate status code.
- Browser: Displays the returned content or handles the response.

For example, when a user enters a website address in a browser, the browser establishes communication with the server and requests the required webpage.

The server may respond with HTML, CSS, JavaScript, images, video content, or other data.

**Example of HTTP communication**

Consider a user visiting a startup's website at http://example.com.

The browser may send the following HTTP request:

```
GET / HTTP/1.1
Host: example.com
Accept: text/html
```

The server may return a response similar to this:

```
HTTP/1.1 200 OK
Content-Type: text/html

<!DOCTYPE html>
<html>
<body>
   <h1>Welcome to Our Startup</h1>
   <p>Our website is working successfully.</p>
</body>
</html>
```

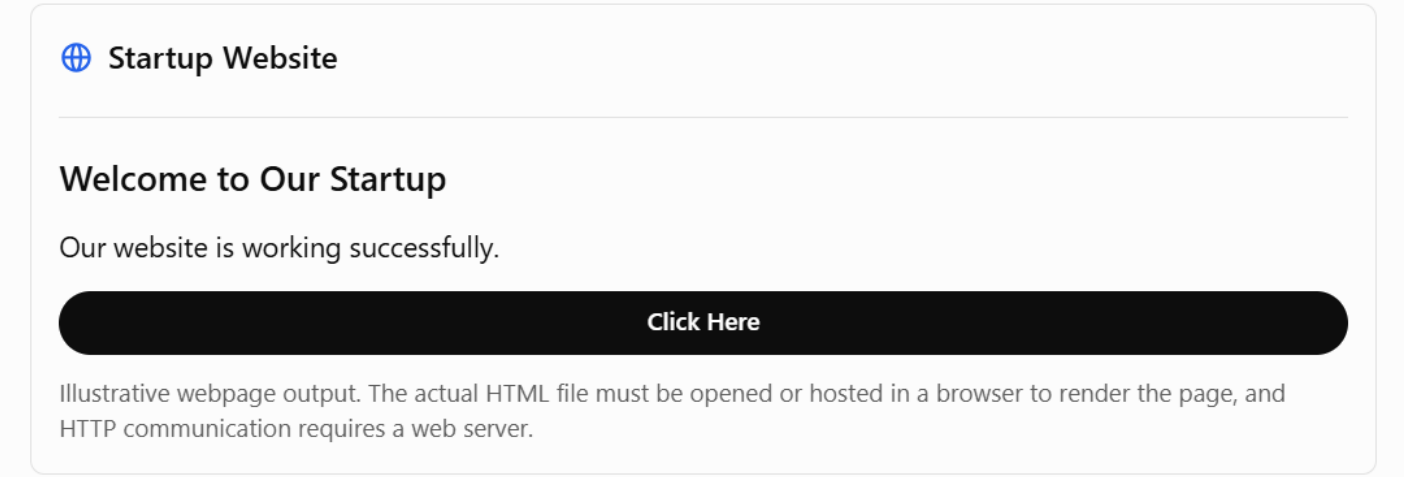
The request asks for the homepage, and the response contains the webpage's HTML.

These are illustrative HTTP messages. A real server's response may contain additional headers and different content.

**HTML code for a simple webpage**

```
<!DOCTYPE html>
<html>
<head>
   <title>Startup Website</title>
</head>
<body>
   <h1>Welcome to Our Startup</h1>
   <p>Our website is working successfully.</p>
   <button onclick="alert('Welcome to our website!')">
      Click Here
   </button>
</body>
</html>
```

**Output**




**Explanation of the HTTP request and response**

Step 1: User enters the website address.

The user enters a website URL in the browser.

Step 2: Browser identifies the server.

The browser uses DNS when necessary to resolve the domain name to an IP address.

Step 3: Connection is established.

For HTTP, the browser establishes the necessary network connection. For HTTPS, it also establishes a TLS-secured connection.

Step 4: Browser sends an HTTP request.

The browser sends a request specifying the resource it needs, such as the homepage.

Step 5: Server processes the request.

The server identifies the requested resource and performs the required operation.

Step 6: Server returns an HTTP response.

The response includes a status code, headers, and, when applicable, a response body containing the webpage or other resource.

Step 7: Browser displays the webpage.

The browser processes the returned HTML and may request additional CSS, JavaScript, images, and other resources.

**Why are the other options incorrect?**

| Option | Protocol | Purpose |
|---|---|---|
| A | FTP | Transfers files between systems |

| | | |
|---|---|---|
| B | HTTP | Transfers web resources between clients and servers |
| C | SMTP | Sends email messages between mail systems |
| D | SNMP | Monitors and manages network devices |

FTP (File Transfer Protocol): FTP is designed for transferring files between computers. It is not the standard protocol for requesting and displaying webpages.

HTTP (Hypertext Transfer Protocol): HTTP is the correct option because it is designed for web communication and supports the exchange of web resources.

SMTP (Simple Mail Transfer Protocol): SMTP is used for sending email messages. It does not normally serve as the protocol for loading webpages.

SNMP (Simple Network Management Protocol): SNMP is used for monitoring and managing network devices, such as routers and switches.

**HTTP versus HTTPS**

| Feature | HTTP | HTTPS |
|---|---|---|
| Full form | Hypertext Transfer Protocol | Hypertext Transfer Protocol Secure |
| Default port | 80 | 443 |
| Encryption | No encryption provided by HTTP itself | Uses TLS to protect communication |
| Data protection | Vulnerable to interception or modification on an untrusted network | Provides confidentiality and integrity in transit |
| Typical usage | Non-sensitive or controlled environments | Modern public websites, logins, and online transactions |

HTTPS is the recommended choice for the startup because its registration form collects user information. However, HTTPS does not automatically protect data stored on the server or prevent every type of application attack; additional security measures are necessary.

**Advantages of HTTP**

1. It enables communication between web browsers and servers.
2. It supports different HTTP methods, including GET and POST.
3. It supports the transfer of HTML, images, videos, stylesheets, and scripts.

4. It provides status codes that indicate the result of requests.
5. It supports headers for content types, caching, authentication, and other communication requirements.
6. It is widely supported by browsers, web servers, and development frameworks.

**Conclusion**

HTTP is the correct answer because it is the standard application-layer protocol used for communication between web browsers and web servers. For the startup's modern web application, HTTPS should be used to secure communication, especially when handling registration information and other sensitive data.